

Katedra informatiky  
Přírodovědecká fakulta  
Univerzita Palackého v Olomouci

# BAKALÁŘSKÁ PRÁCE

Procedurální generování dungeonů ve 2D hrách

Implementace a srovnání algoritmů v herním enginu Godot



2026

Vedoucí práce:  
doc. RNDr. Jan Konečný, Ph.D.

Jesse Sadowý

Studijní program: Informační technologie,  
kombinovaná forma

## Bibliografické údaje

Autor: Jesse Sadowý

Název práce: Procedurální generování dungeonů ve 2D hrách (Implementace a srovnání algoritmů v herním enginu Godot)

Typ práce: bakalářská práce

Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci

Rok obhajoby: 2026

Studijní program: Informační technologie, kombinovaná forma

Vedoucí práce: doc. RNDr. Jan Konečný, Ph.D.

Počet stran: 22

Přílohy: test

Jazyk práce: český

## Bibliographic info

Author: Jesse Sadowý

Title: Procedural Generation of Dungeons in 2D Games (Implementation and Comparison of Algorithms in Godot Game Engine)

Thesis type: bachelor thesis

Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc

Year of defense: 2026

Study program: Information Technologies, combined form

Supervisor: doc. RNDr. Jan Konečný, Ph.D.

Page count: 22

Supplements: electronic data in the storage of department of computer science

Thesis language: Czech

## Anotace

*10-20 řádků Bakalářská práce se zabývá procedurálním generováním dungeonů ve 2D hrách. Zkoumá různé metody a techniky používané pro generování a následně porovnává jejich efektivitu a výkonnost v prostředí herního enginu Godot.*

## Synopsis

*Bachelor's thesis focuses on procedural generation of dungeons in 2D games. It explores various methods and techniques used for generation and subsequently compares their efficiency and performance within the Godot game engine environment.*

**Klíčová slova:** procedurální generování; dungeon; 2D hry; Godot; herní engine, algoritmy

**Keywords:** procedural generation; dungeon; 2D games; Godot; game engine, algorithms

Děkuji, děkuji, děkuji.

*Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.*

# Obsah

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Úvod</b>                                                       | <b>8</b>  |
| <b>2</b> | <b>Procedurální generování obsahu ve hrách</b>                    | <b>8</b>  |
| 2.1      | Vymezení a definice pojmu . . . . .                               | 8         |
| 2.2      | Motivace a důvody využití procedurálního generování . . . . .     | 8         |
| 2.3      | Nevýhody procedurálního generování . . . . .                      | 9         |
| 2.4      | Přístupy k procedurálnímu generování . . . . .                    | 9         |
| 2.5      | Využití procedurálního generování v herním průmyslu . . . . .     | 10        |
| 2.6      | Rouglíky hry a jejich vztah k procedurálnímu generování . . . . . | 10        |
| 2.6.1    | Beneath Apple Manor . . . . .                                     | 11        |
| 2.6.2    | Rogue . . . . .                                                   | 11        |
| 2.6.3    | Elite . . . . .                                                   | 12        |
| <b>3</b> | <b>Generování dungeonů ve 2D hrách</b>                            | <b>13</b> |
| 3.1      | Požadavky na generování dungeonových map . . . . .                | 13        |
| 3.2      | Přehled vybraných metod generování . . . . .                      | 14        |
| 3.2.1    | Generování pomocí náhodného umístění místností a chodeb . . . . . | 15        |
| 3.2.2    | Rozdělování prostoru metodou Binary Space Partitioning . . . . .  | 15        |
| 3.2.3    | Generování pomocí buněčných automatů . . . . .                    | 15        |
| 3.2.4    | Metoda náhodné procházky . . . . .                                | 15        |
| 3.2.5    | Metoda bludiště . . . . .                                         | 15        |
| 3.2.6    | Další metody . . . . .                                            | 15        |
| <b>4</b> | <b>Návrh a použité technologie</b>                                | <b>15</b> |
| 4.1      | Technologie . . . . .                                             | 15        |
| 4.1.1    | Herní engine Godot . . . . .                                      | 15        |
| 4.1.2    | Skriptovací jazyk GDScript . . . . .                              | 15        |
| 4.2      | Návrh mapy a hráče . . . . .                                      | 15        |
| <b>5</b> | <b>Implementace generovacích metod</b>                            | <b>15</b> |
| 5.1      | Generování pomocí místností a chodeb . . . . .                    | 15        |
| 5.2      | Generování pomocí bludiště . . . . .                              | 15        |
| 5.3      | Binary Space Partitioning . . . . .                               | 15        |
| 5.4      | Metoda náhodné procházky (Drunkard's Walk) . . . . .              | 16        |
| 5.5      | Metoda buněčných automatů . . . . .                               | 16        |
| 5.6      | Další možné metody generování . . . . .                           | 16        |
| 5.7      | Možnosti budoucí implementace . . . . .                           | 16        |
| <b>6</b> | <b>Vyhodnocení a porovnání metod</b>                              | <b>16</b> |
| 6.1      | Hodnoticí kritéria . . . . .                                      | 16        |
| 6.2      | Porovnání výsledných map . . . . .                                | 16        |
| 6.3      | Výpočetní náročnost metod . . . . .                               | 16        |
| 6.4      | Shrnutí výsledků . . . . .                                        | 16        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>7</b> | <b>Diskuze</b>                                     | <b>16</b> |
| 7.1      | Zkušenosti z implementace . . . . .                | 16        |
| 7.2      | Silné a slabé stránky jednotlivých metod . . . . . | 16        |
| 7.3      | Možnosti dalšího rozšíření práce . . . . .         | 16        |
|          | <b>Závěr</b>                                       | <b>17</b> |
|          | <b>Conclusions</b>                                 | <b>18</b> |
| <b>A</b> | <b>Ukázky zdrojového kódu</b>                      | <b>19</b> |
| <b>B</b> | <b>Odkaz na repozitář projektu</b>                 | <b>19</b> |
| <b>C</b> | <b>Zkratky</b>                                     | <b>19</b> |
| <b>D</b> | <b>Obsah elektronických dat</b>                    | <b>19</b> |
|          | <b>Seznam zkratk</b>                               | <b>21</b> |
|          | <b>Literatura</b>                                  | <b>22</b> |

## Seznam tabulek

# 1 Úvod

this section

## 2 Procedurální generování obsahu ve hrách

### 2.1 Vymezení a definice pojmu

Procedurální generování obsahu ve hrách lze vymezit jakožto algoritmickou tvorbu herního obsahu s limitovaným nebo nepřímým zásahem člověka. Tento přístup nám umožňuje automatizovanou tvorbu různých prvků hry bez nutnosti jejich ruční úpravy. Herní obsah zahrnuje široké spektrum prvků jako jsou mapy, úrovně, pravidla, úkoly, zbraně či předměty.

PCG se tedy neomezuje pouze na herní prostřední, ale může být aplikováno i na další složky herního systému. V závislosti na konkrétním návrhu může ovlivnit nejen prostorovou strukturu hry, ale také její mechaniky, obtížnost či samotný průběh hraní.

V současné době je pojem generování obsahu často spojován s využitím metod umělé inteligence, avšak v této práci se pojednává výhradně o algoritmické metody, založené na předem definovaných pravidlech a postupech, nikoli na adaptivním či učícím se chování systémů umělé inteligence.

Klíčovým pojmem v této souvislosti jsou samotné hry, jejichž definice je dosti problematická. Pro účely této práce se jedná především o digitální videohry určené pro osobní počítače, případně další platformy

Termíny "procedurální" a "generování" zdůrazňují skutečnost, že se jedná o proces realizovaný prostřednictvím počítačových procedur a algoritmů, jejichž spuštění zahájí člověk, ale výsledkem procesu je automaticky vytvoření herní obsah, který není pevně předem definován, ale vzniká za běhu programu.

### 2.2 Motivace a důvody využití procedurálního generování

Hlavním důvodem pro použití PCG je, že obsah je v podstatě generován sám, takže odstraňuje nutnost kreslit každou novou mapu či položku zvlášť, vytvářet logiku pro další úrovně, případně v každé truhle, kterou ve hře najdeme, specifikovat, co tam bude. Nejenže tím zjednodušíme tvorbu, ale výrazně ušetříme čas i náklady na vývoj dané hry.

Dalším výrazným přínosem je to, že PCG přidává momenty překvapení a znovuhratelnosti, jelikož obsah se bude při každém průchodu lišit a hráč tím pádem nikdy neví, co se stane a jaká výzva ho čeká. Dále nám to umožňuje hru přizpůsobit samotnému hráči podle toho, jak hraje. Příkladem může být proměnlivá obtížnost, která zůstane vyvážená a hra tím nebude příliš jednoduchá či složitá. Vhodné je použít PCG i při umísťování herních předmětů či nepřátel, tím pádem bude každý průchod jedinečný a opět to dodá hře unikátnost a znovuhratelnost,



jelikož hráč se nemohl spoléhat na zapamatování si rozložení herního světa z předhcozích průchodů.

Na první pohled to tak nemusí vypadat, ale tento způsob dodává hře i více kreativních prvků, jelikož daný algoritmus může přinést výsledky, na které by vývojář nemusel nikdy přijít. Člověk má tendenci používat to, co už někdy viděl a tímto bude obsah tvořen zcela novým a nečekaným způsobem.

Nakonec je vhodné podotknout, že díky PCG se naučíme porozumět i designu samotných her. Jak uvádí Shaker et al. [**pcgbook**], proces tvorby generátoru vyžaduje hluboké porozumění herním mechanikám a principům dobrého designu, které musíme převést do algoritmické podoby.

## 2.3 Nevýhody procedurálního generování

Přestože procedurální generování obsahu přináší řadu výhod, jeho využití s sebou nese také několik významných omezení a úskalí.

Jedním z hlavních problémů je nižší míra kontroly nad výsledným obsahem. U ručně navržených úrovní má vývojář plnou kontrolu nad tempem hry, rozmístěním výzev, předmětů i klíčových herních momentů. Naproti tomu u procedurálního generování je výsledná podoba světa závislá na implementovaném algoritmu a zvolených parametrech. To může vést k nevyváženým úrovním, nelogickému rozmístění prvků nebo situacím, které jsou příliš jednoduché či naopak extrémně náročné. Vývojář proto musí věnovat značné úsilí návrhu, ladění a testování generátoru, aby bylo dosaženo uspokojivé kvality výsledného obsahu.

Dalším rizikem je možnost vzniku repetitivního nebo uměle působícího obsahu. Ačkoli je obsah generován automaticky, často vychází z omezené množiny předem definovaných prvků a pravidel. Pokud není systém navržen dostatečně variabilně, může hráč postupně rozpoznat opakující se vzorce, což může negativně ovlivnit herní zážitek. Tento problém je sice možné minimalizovat vhodným návrhem algoritmu a důkladným laděním, nicméně zcela eliminovat jej bývá obtížné.

## 2.4 Přístupy k procedurálnímu generování

Procedurální generování obsahu lze rozdělit podle několika hledisek. Jedním z nich je systém reaguje na hráče. V případě neadaptivního (statického) generování je obsah vytvářen bez ohledu na chování hráče a jeho průběh hry. Všechny generované struktury vycházejí ze stejných pravidel.

Naopak adaptivní generování pracuje s informacemi o hráči. Systém může sledovat například úspěšnost hráče, délku průchodu úrovní nebo způsob řešení situací a na základě těchto dat upravovat další generovaný obsah.

Dalším důležitým rozdělením je deterministický a stochastický přístup. Deterministické generování vychází z pevně daných počátečních hodnot (například seed) a při stejném vstupu vždy produkuje stejný výstup. Tento přístup umožňuje reprodukovatelnost výsledků, což je výhodné například při testování nebo

sdílení konkrétních map mezi hráči. Stochastické generování naproti tomu pracuje s náhodností bez pevně stanoveného počátečního stavu, takže výsledky se mohou lišit i při opakovaném spuštění algoritmu.

Z hlediska samotného procesu generování lze rozlišit konstruktivní metody a přístup označovaný jako generate-and-test. Konstruktivní metody vytvářejí obsah postupně podle předem definovaných pravidel a omezení. Každý krok generování přímo přispívá k vytvoření výsledné struktury. Naproti tomu metoda generate-and-test nejprve vytvoří potenciální řešení, které je následně ověřeno podle stanovených kritérií. Pokud výsledek nevyhovuje, proces se opakuje. Tento přístup může být výpočetně náročnější, ale umožňuje lépe kontrolovat výsledné vlastnosti mapy.

V literatuře se rovněž objevuje pojem mixed-authorship (smíšené autorství), kdy se na tvorbě obsahu podílí jak algoritmus, tak lidský návrhář. Systém může například generovat návrhy úrovní, které následně autor upravuje, případně může reagovat na částečně ručně vytvořenou strukturu. Tento přístup kombinuje kreativitu člověka s rychlostí automatizovaného generování.

Plně automatická generace naopak probíhá bez zásahu člověka během samotného procesu vytváření obsahu. Algoritmus pracuje pouze na základě vstupních parametrů a předem definovaných pravidel. Tento způsob je typický zejména pro roguelike hry a další tituly, kde je cílem vysoká různost jednotlivých průchodů.

## 2.5 Využití procedurálního generování v herním průmyslu

V dnešní době se s procedurálním generováním setkáváme téměř v každé moderní videohře, avšak jeho počátky spadají již do 70. let 20. století. V tomto období byl výpočetní výkon, a zejména dostupná paměť RAM, velmi omezený. Tyto limity omezovaly množství ručně vytvářeného herního obsahu a vývojáři tedy museli hledat způsoby, jak tato omezení překonat a jak implementovat variabilní herní prostředí s minimálním nárokem na paměť. A právě tady se začaly prosazovat procedurální přístupy.

## 2.6 Roguelike hry a jejich vztah k procedurálnímu generování

V kontextu procedurálního generování je nutné zmínit i pojem roguelike, který označuje specifický herní subžánr úzce spojený s náhodně generovaným herním obsahem. Název žánru je odvozen od hry Rouge, která významně ovlivnila podobu mnoha pozdějších titulů.

Roguelike hry jsou často charakteristické procedurálně generovanými úrovněmi, tahovým systémem hry, permanentní smrtí postavy (permadeath), a vysokou mírou obtížnosti. Náhodné generování prostředí zde hraje klíčovou roli neboť představuje unkinátní herní zážitek při každém průchodu. Dále se jedná i o významný designový prvek, který podporuje průzkum, strategické rozhodování a adaptaci hráče na neustále se měnící prostředí. Právě díky těmto vlastnostem se

roguelike hry staly jedním z nevýraznějších příkladů praktického a systematického generování v herním průmyslu.

### 2.6.1 Beneath Apple Manor

Jednou z prvních her, která PCG implementovala, se objevila už v roce 1978 pod názvem Beneath Apple Manor. Za tímto titulem stojí Don Worth a vydání zajistilo herní studio The Software Factory, hra byla primárně vyvíjena pro počítač Apple II.

Herní prostředí v Beneath Apple Manor bylo generováno algoritmicky při každém novém spuštění hry. Tento přístup zajistil, že rozložení místností, chodeb a jednotlivých herních prvků či nepřátel nebolo nikdy totožné, čímž se výrazně zvětšila znovuhratelnost. Cílem hry bylo získat zlaté jablko a progres bylo možné ukládat, takže při smrti hráče se načetla uložená úroveň znovu s tím, že hráč přišel o nějaké atributy.

Autor hry čerpal inspiraci z programu Dragon Maze, který rovněž využíval náhodné výpočty ke generování bludišť. Jeho vlastní algoritmus však tento princip dále rozvinul a navrhl algoritmus, který vytvářel funkční a hratelnou strukturu dungeonů skládající se s místnosti propojených chodbami. Hra navíc umožňovala hráči ovlivnit některé parametry samotného generování, jako například nastavit počet místností na jednotlivých patrech, zvolit si variantu černobílou nebo barvenou, a nakonec i obtížnost. Po vygenerování struktury úrovně algoritmus náhodně rozmístil i nepřátele, poklady či další herní objekty. S rostoucí hloubkou dungeonů se zároveň i zvyšovala obtížnost hry, a to zejména síla nepřátel.

Použitý algoritmický přístup lze zařadit mezi rané příklady způsobu generování založených na místnostech a chodbách, které se později stalo jedním ze základních modelů využívaných v hrách typu roguelike. Beneath Apple Manor tak představuje významný historický příklad využití těchto metod, jehož principy byly dále rozvíjeny v následujících dekáдах.

### 2.6.2 Rogue

Za zmínku stojí i herní titul s názvem Rogue, který si oproti výše zmíněnému Beneath Apple Manor získal výrazně větší popularitu a měl zásadní vliv na další vývoj herního designu. Právě podle této hry vznikl podžánr označovaný jako roguelike, jenž z jejích principů vychází dodnes.

Hra byla představena v roce 1980 a jejími autory jsou Michael Toy, Glenn Wichman a Ken Arnold. Původně byla vyvíjena pro unixové systémy a zobrazovala herní svět pomocí ASCII znaků v textovém terminálu. Jednotlivé prvky prostředí byly reprezentovány znaky, například hráč symbolem @, stěny znakem #, nepřátele různými písmeny abecedy. Tento způsob zobrazení nebyl pouze důsledkem technických omezení tehdejšího hardwaru, ale zároveň umožňoval poměrně flexibilní práci s herním světem a jeho generováním.

Rogue je obvykle popisováno jako jedna z prvních roguelike adventur, která výrazně zpopularizovala princip permanentní smrti (permadeath). Postup hrou

nebylo možné ukládat a jakmile hráč zemřel, musel začít zcela od začátku. Hráči tak zůstávaly pouze nabyté zkušenosti a znalost herních mechanik. Smrt tedy nepředstavovala jen neúspěch, ale i součást učení se systému hry.

Cílem hry je najít Amulet of Yendor, který se nachází v nejhlubším patře dungeonů, a následně se vrátit zpět na povrch. Dungeon je tvořen několika patry (tradičně 26), přičemž každé z nich je generováno procedurálně. Patra se skládají z místností propojených chodbami a jejich rozložení se při každém novém průchodu mění. Náhodně jsou rozmístěni také nepřátelé, pasti, poklady, zbraně a další předměty.

Prvek náhody se projevuje i ve vlastnostech předmětů. Například lektvary, svitky či prsteny mají při každé nové hře odlišné označení a jejich účinek není hráči předem znám. I pokud hráč nalezne stejný typ předmětu jako v předchozí hře, jeho vlastnosti se mohou lišit. Tím je posílena nejistota a nutnost experimentování.

Kombinace procedurálně generovaných úrovní a permanentní smrti zajišťuje, že každý průchod hrou je unikátní. Hráč se tak nemůže spoléhat na zapamatování konkrétní mapy, ale musí se přizpůsobovat aktuálně vygenerovaným podmínkám. Právě tento princip se stal jedním ze základních stavebních kamenů celého roguelike žánru.

### 2.6.3 Elite

Dalším významným titulem využívajícím procedurální generování je hra Elite z roku 1984, kterou vytvořili David Braben a Ian Bell. Na rozdíl od předchozích her se nejedná o dungeon, ale o vesmírný simulátor obchodování a soubojů. Přesto je Elite z hlediska procedurálního generování velmi důležitým milníkem.

Hra byla původně vydána pro počítač BBC Micro, který disponoval velmi omezenou pamětí. Z tohoto důvodu nebylo možné ukládat rozsáhlý herní svět přímo v paměti. Vývojáři proto zvolili přístup, kdy byl herní svět generován algoritmicky na základě předem definovaného číselného základu (seed). Pomocí tohoto deterministického postupu bylo možné při každém spuštění znovu vytvořit stejnou galaxii bez nutnosti jejího explicitního uložení.

Elite obsahovalo osm galaxií, přičemž každá z nich zahrnovala stovky hvězdných systémů. Vlastnosti jednotlivých systémů, jako je jejich název, ekonomika, úroveň technologického rozvoje či typ vlády, byly generovány pomocí pseudonáhodných algoritmů. Procedurálně byly generovány také ceny komodit a další herní parametry.

Tento přístup ukázal, že procedurální generování není omezeno pouze na tvorbu dungeonů nebo jednotlivých úrovní, ale může být využito i pro vytvoření rozsáhlých herních světů. Elite tak demonstrovalo, že algoritmická generace obsahu představuje efektivní řešení zejména v podmínkách omezených výpočetních a paměťových zdrojů.

## 3 Generování dungeonů ve 2D hrách

V předchozí kapitole byly popsány obecné principy procedurálního generování obsahu a jeho využití v herním průmyslu. Tato kapitola se zaměřuje konkrétně na generování dungeonových map ve 2D hrách, které představují jednu z nejčastějších aplikací procedurálního přístupu.

Dungeonové mapy mají svá specifika, která je odlišují od jiných typů herních prostředí. Typicky se skládají z místností a chodeb, obsahují jasně definované průchozí a neprůchozí oblasti a musí splňovat určité požadavky na hratelnost, jako je průchodnost celé mapy nebo vhodné rozmístění herních prvků. Generování takových struktur proto vyžaduje algoritmy, které dokáží zajistit nejen variabilitu výsledku, ale i splnění základních strukturálních vlastností.

V této kapitole budou nejprve definovány požadavky kladené na generování dungeonových map. Následně budou představeny vybrané metody generování, které se v praxi používají, a které tvoří teoretický základ pro implementaci popsanou v dalších kapitolách.

### 3.1 Požadavky na generování dungeonových map

Při návrhu algoritmu pro generování dungeonové mapy je nutné zohlednit několik základních požadavků, které vycházejí jak z technických omezení, tak z hlediska hratelnosti. Generovaná mapa by neměla být pouze náhodným shlukem místností a chodeb, ale strukturovaným prostředím, které umožňuje smysluplný průchod a interakci.

Jedním ze základních požadavků je průchodnost mapy (connectivity). Všechny klíčové části mapy by měly být vzájemně dosažitelné, aby nedocházelo k izolovaným oblastem bez možnosti přístupu. Tento požadavek je zásadní zejména u her, kde je cílem projít celou úroveň nebo nalézt konkrétní objekt.

Dalším důležitým aspektem je kontrola struktury a rozložení prostoru. Mapa by měla obsahovat přiměřený poměr otevřených a uzavřených prostor, vhodnou velikost místností a dostatečně široké průchody aby hráč mohl bez problémů projít. Příliš husté nebo naopak příliš prázdné prostředí může negativně ovlivnit hratelnost i orientaci hráče.

Významným požadavkem je rovněž variabilita výsledků. Opakované spuštění generátoru by mělo vést k odlišným mapám, aby byl zachován prvek znovuhratelnosti. Zároveň je však vhodné, aby generování respektovalo určitá pravidla nebo omezení, která zajistí konzistentní kvalitu výsledku.

Z technického hlediska je důležitá také výpočetní náročnost algoritmu. Generování by mělo probíhat v přiměřeném čase a nemělo by výrazně zatěžovat systém, zejména pokud je realizováno za běhu hry (tzv. online generování). U jednodušších 2D dungeonových her je obvykle možné využít algoritmy s relativně nízkou časovou složitostí.

V neposlední řadě je vhodné umožnit parametrizaci generátoru. Možnost nastavit velikost mapy, počet místností, hustotu chodeb nebo další vlastnosti umož-

ňuje lépe přizpůsobit výsledné prostředí konkrétním požadavkům hry a zároveň usnadňuje testování různých konfigurací.

### 3.2 Přehled vybraných metod generování

Na základě výše uvedených požadavků a pravidel lze identifikovat několik přístupů a algoritmů generování dungeonových map. Tyto metody se liší zejména způsobem konstrukce prostoru, mírou kontroly, výslednou strukturou a výpočetní náročností. Některé algoritmy vytvářejí mapu postupně přidáváním jednotlivých prvků, jiné pracují s rozdělováním prostoru nebo s pravidly aplikovanými na mřížku buněk.

Ve 2D hrách se nejčasteji setkáváme s metodami založenými na náhodném umísťování místností a následném propojování, s rozdělováním velkého prostoru na menší a menší části či s algoritmy založenými na takzvané náhodné procházce. Každý z těchto přístupů má své výhody i omezení a je vhodný pro odlišný typ herního prostředí.

Cílem této podkapitoly je stručně představit princip fungování vybraných metod, jejich základní charakteristiky a typické oblasti využití. Detailnější rozpracování a samotná implementace vybraných algoritmů bude popsána v následujících kapitolách.

- 3.2.1 Generování pomocí náhodného umístění místností a chodeb
- 3.2.2 Rozdělování prostoru metodou Binary Space Partitioning
- 3.2.3 Generování pomocí buněčných automatů
- 3.2.4 Metoda náhodné procházky
- 3.2.5 Metoda bludiště
- 3.2.6 Další metody

## 4 Návrh a použité technologie

### 4.1 Technologie

- 4.1.1 Herní engine Godot
- 4.1.2 Skriptovací jazyk GDScript

### 4.2 Návrh mapy a hráče

## 5 Implementace generovacích metod

- 5.1 Generování pomocí místností a chodeb
- 5.2 Generování pomocí bludiště
- 5.3 Binary Space Partitioning

5.4 Metoda náhodné procházky (Drunkard's Walk)

5.5 Metoda buněčných automatů

5.6 Další možné metody generování

5.7 Možnosti budoucí implementace

## 6 Vyhodnocení a porovnání metod

6.1 Hodnoticí kritéria

6.2 Porovnání výsledných map

6.3 Výpočetní náročnost metod

6.4 Shrnutí výsledků

## 7 Diskuze

7.1 Zkušenosti z implementace

7.2 Silné a slabé stránky jednotlivých metod

7.3 Možnosti dalšího rozšíření práce



## Závěr

## Conclusions

## A Ukázky zdrojového kódu

## B Odkaz na repozitář projektu

## C Zkratky

PCG - Procedural Content Generation

## D Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

### **text/**

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

### **README . \***

Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

### **\***

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,

- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwareového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.\*.



## Literatura

- [1] Shaker, Noor; Togelius, Julian; Nelson, Mark J. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. 2016. 247 s. ISBN 978-3-319-42714-0.
- [2] Linden, Roland; Lopes, Ricardo; Bidarra, Rafael. Procedural Generation of Dungeons. *Computational Intelligence and AI in Games, IEEE Transactions on*. 2014, roč. 6, s. 78–89. Dostupný také z: <http://dx.doi.org/10.1109/TCIAIG.2013.2290371>.
- [3] Wikipedia Contributors. *Elite (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 22-February-2026]. Dostupný z: [https://en.wikipedia.org/wiki/Elite\\_\(video\\_game\)](https://en.wikipedia.org/wiki/Elite_(video_game)).
- [4] Procedural Content Generation Wiki. *Rogue* [online]. 2016 [cit. 2024-5-22]. Dostupný z: <http://pcg.wikidot.com/pcg-games:rogue>.
- [5] Wikipedia Contributors. *Rogue (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 22-February-2026]. Dostupný z: [https://en.wikipedia.org/w/index.php?title=Rogue\\_\(video\\_game\)&oldid=1337873899](https://en.wikipedia.org/w/index.php?title=Rogue_(video_game)&oldid=1337873899).
- [6] Schuller, Dan. *Dive into beneath Apple Manor: A pre-rogue roguelike*. 2021. Dostupný z: <https://howtomakeanrpg.com/r/l/g/apple-manor.html>.
- [7] Wikipedia Contributors. *Beneath Apple Manor* — *Wikipedia, The Free Encyclopedia*. 2024. Dostupný z: [https://en.wikipedia.org/w/index.php?title=Beneath\\_Apple\\_Manor&oldid=1260179812%22](https://en.wikipedia.org/w/index.php?title=Beneath_Apple_Manor&oldid=1260179812%22).